Econometrics Midterm

R Functions · W1, W2, W5, W6 · 31 questions

1. Read the code. What does table(x) return, and what does sort(..., decreasing = TRUE) do with it?

```
x <- c(2, 3, 3, 5, 3, 2)
table(x)
sort(table(x), decreasing = TRUE)
```

2. Read the code from Lab2.R. What does read.csv() do, and what do the arguments header = TRUE and na.strings = "." do?

```
practice <- read.csv("Lab2_Practice.csv", header=TRUE, na.strings=".")
```

3. Read the code from Lab2.R. What do attach(practice) and detach(practice) do?

```
attach(practice)
mean(COMPETENCE)
detach(practice)
```

4. For breaks, the first line returns 0, 1, 2, 4.5 and 12 and the second returns 3.5. What does each function compute?

```
quantile(breaks)
IQR(breaks)
```

5. Read the code from Lab2.R. What does boxplot(grade) draw, and what does abline(h = median(grade), col = "red") add to it?

```
boxplot(grade)
abline(h=median(grade), col="red")
```

6. Read the code from Lab2.R. What does vioplot() draw, and why must install.packages("vioplot") and library("vioplot") be run first?

```
install.packages("vioplot")
library("vioplot")
vioplot(competence, breaks, grade)
```

7. Read the code from Lab2.R. What does par(mfrow = c(1, 2)) do to the plots that follow it?

```
par(mfrow = c(1, 2))
boxplot(grade)
vioplot(grade)
```

8. What does source() do, and what does the argument echo = TRUE add?

```
source("~/R_w2.R", echo = TRUE)
```

9. Read the code from Lab2.R (competence has 27 values). What does the first line compute, step by step, and which built-in function gives the same value?

```
sum((competence-mean(competence))^2)/(length(competence)-1)
var(competence)
```

10. In mtcars, am = 0 marks cars with an automatic transmission. What does mpg[am == 0] select, and what does the whole line compute?

```
attach(mtcars)
mean(mpg[am==0])
```

11. Read the function from Lab4.R. What does to12() do with its input, and what do as.integer() and paste() contribute to it?

```
to12 <- function(str){
  hour=as.integer(str)
  if (hour>12) {
    return(paste(hour-12, "PM"))
  } else {
    return(paste(hour, "AM"))
  }
}
to12("22")
```

12. In Lab4.R, hour = "13" is stored as text. Why does paste(hour - 12, "PM") stop with an error, and how does to12() avoid the problem?

```
hour="13"
if(as.integer(hour)>12) {
  print(paste(hour-12, "PM"))
}
```

13. Read the code from Lab4.R. What does str_detect(str, fixed(",")) return, what does fixed() do, and what do the two calls return?

```
library(stringr)
hasMultipleLoc <- function(str){
  str_detect(str, fixed(","))
}
hasMultipleLoc("Seoul, Suwon")
hasMultipleLoc("Seoul")
```

14. Read the code from Lab4.R. What does sapply() do here, and why is it used instead of calling to12(data$openingHour) directly?

```
(openingHour <- sapply(data$openingHour, to12))
```

15. Read the code from Lab4.R. What does ifelse() do in this line, and what ends up in the new column?

```
(locationText <- ifelse(sapply(data$location, hasMultipleLoc), "Yes", "No"))
data$hasMultipleLoc = locationText
```

16. Read the code from Lab4.R. What does each of the three functions do?

```
library(readxl)
data = read_excel("Café.xlsx")
data <- as.data.frame(data)
colnames(data) = c("type", "location", "openingHour", "closingHour")
```

17. Read the code from regression.R. What does lm() return here, and what does abline() draw when it is given that result?

```
plot(perform ~ eye)
eye_regr = lm(perform ~ eye)
abline(eye_regr, col = "blue")
```

18. Read the code from regression.R. What does summary() report for an lm model, and what does $r.squared extract?

```
eye_summary = summary(eye_regr)
eye_summary
eye_summary$r.squared
```

19. Read the code from regression.R. What does confint() return, and what changes when level = 0.99 is added?

```
confint(eye_regr)
confint(eye_regr, level = 0.99)
```

20. Read the code from regression.R. What does cor() compute, why is it squared, and what is round(..., 16) for?

```
(skill_coef = cor(perform, skill)^2)
round(skill_summary$r.squared - skill_coef, 16)
```

21. Read the code from regression.R. What does predict() do with the data frame it is given, and what do the arguments interval and level change?

```
predict(mult_lm, data.frame(skill=c(0.5,1.5), ravens=c(1.5,0.5)),
interval="confidence", level=0.95)
```

22. Read the code from regression.R. What does each of seq(), polygon() and lines() do in drawing the confidence band?

```
points <- seq(min(eye)-3, max(eye)+3, length.out=10000)
pred <- predict(eye_regr, data.frame(eye=points), interval="confidence")
polygon(c(rev(points), points), c(rev(pred[ ,3]), pred[ ,2]), col=rgb(0,0,0,0.1),
border=NA)
lines(points, pred[ ,3], lty='dashed', col='green')
```

23. Read the code from the simulation script. What do set.seed() and rnorm() do?

```
set.seed(4201)
fertilizer = rnorm(sampleSize, 10, 2)
yield = intercept + slope*fertilizer + rnorm(sampleSize, 0, 1)
```

24. Read the simulation loop (sampleCnt = 10,000). What does the for loop do, and what do coef[2,1], coef[2,3] and coef[2,4] contain?

```
for(rowIdx in 1:sampleCnt) {
  fertilizer=rnorm(sampleSize,10,2)
  yield=intercept+slope*fertilizer+rnorm(sampleSize,0,1)
  coef=summary(lm(yield~fertilizer))$coefficients
  estimator[rowIdx]=coef[2,1]
  t[rowIdx]=coef[2,3]
  pValue[rowIdx]=coef[2,4]
}
```

25. Read the code from the simulation script. What does each line do, and what does the share of "Reject" estimate here (the true slope is 0.2)?

```
rejection = pValue <= alpha
rejection = ifelse(rejection, "Reject", "Fail to Reject")
(result = table(rejection))
barplot(result, main = "Null Hypothesis Rejection")
```

26. Read the confidence-interval loop. What do data.frame(), results[rowIdx, "slope"], interval["fertilizer", "2.5 %"] and && do?

```
results = data.frame(index=1:sampleCnt)
results[rowIdx,"slope"] = fer_regr$coefficients[2]
interval = confint(fer_regr)
lowerBound = interval["fertilizer","2.5 %"]
upperBound = interval["fertilizer","97.5 %"]
inside = lowerBound<=slope && slope<=upperBound
```

27. Read the plotting code from the simulation script. What does each layer add to the chart of the 20 confidence intervals?

```
ggplot(results, aes(x=slope, y=index)) + geom_point(size=3.5) +
  geom_errorbarh(aes(xmin = lower_bound, xmax = upper_bound)) +
  geom_vline(xintercept = 2, color = "green")
```

28. What does as.factor(session) do in this regression, and what does lm() then create from it?

```
model <- lm(y ~ x + as.factor(session), data = df)
```

29. What terms does each of the three formulas put into the model?

```
lm(y ~ x1:x2)
lm(y ~ x1*x2)
lm(y ~ x1 + x2)
```

30. What does linearHypothesis() from the car package do with the model in this call?

```
library(car)
linearHypothesis(model, c("x1 = 0", "x2 = 0"))
```

31. What does anova(restricted, unrestricted) do, and what test is it here?

```
restricted <- lm(wage ~ educ + exper, data = df)
unrestricted <- lm(wage ~ (educ + exper)*female, data = df)
anova(restricted, unrestricted)
```

Econometrics Midterm — Model answers

1. W1 · Finding the mode with table() and sort()
table(): counts how many times each distinct value occurs and returns a **frequency** table (here 2 twice, 3 three times, 5 once)
sort(): with decreasing = TRUE orders those counts from largest to smallest (**descending**)
The first entry is the value with the highest count, the **mode** (here 3)
Key results: frequency · decreasing · mode

2. W2 · read.csv() and its arguments
read.csv(): reads a comma-separated text file and returns a **data frame**
header: TRUE means the **first row** holds the column (variable) names, not data
na.strings: every cell that contains "." is read as **NA** (missing), so it does not enter calculations as a value
Several codes can be given at once, for example c(".", "-")
Key results: data frame · first row · NA

3. W2 · attach() and detach()
attach(): adds the data frame to R's **search path**, so its columns can be used **by name** (COMPETENCE instead of practice$COMPETENCE)
detach(): **removes** the data frame from the search path again
Risk: a column can mask, or be masked by, another object with the same name
Key results: search path · by name · removes

4. W2 · quantile() and IQR()
quantile(): by default returns the 0%, 25%, 50%, 75% and 100% points, that is the **minimum**, the first **quartile**, the median, the third quartile and the maximum (0, 1, 2, 4.5, 12)
IQR(): the interquartile range, the **third quartile minus** the first quartile (4.5 - 1 = 3.5), the spread of the middle 50% of the data
Key results: quartile · minimum · third quartile minus

5. W2 · boxplot() and abline(h =)
boxplot(): draws a **box** from the lower to the upper hinge (about Q1 to Q3) with a bold line at the **median**
The **whiskers** reach the most extreme points within 1.5 times the IQR of the box, and points beyond them are drawn as outliers
abline(): with h set to a value it draws a **horizontal** line at that y-value, so here a red line through the median
Key results: box · median · whiskers · horizontal

6. W2 · vioplot() and its package
vioplot(): draws a violin plot for each variable, a smoothed **density** mirrored on both sides with a small **box plot** inside
install.packages(): downloads the **package** once, because vioplot is not part of base R
library(): **loads** the package in every new session so that vioplot() can be found
Key results: density · box plot · package · load

7. W2 · par(mfrow =)
par(): sets **graphics parameters**
mfrow: c(1, 2) splits the plotting window into **1 row and 2 columns**, and the following plots fill the panels row by row
Here boxplot(grade) and vioplot(grade) appear **side by side**
The layout stays in effect **until** it is reset, for example with par(mfrow = c(1, 1))
Key results: graphics parameter · 1 row and 2 columns · side by side · until

8. W2 · source() with echo = TRUE
source(): **runs** every command saved in the script file, from top to bottom
echo: TRUE also **prints** each command in the console before its output, as if it had been typed there
Key results: runs · prints

9. W2 · Sample variance written out
(competence - mean(competence))^2: squares every deviation from the mean, and sum() adds them up (about 124.07)
length(): returns n = 27, so the sum is divided by **n - 1** = 26
The result, about 4.77, is the **sample variance**, the same value that var(competence) returns
Key results: sample variance · n - 1 · var

10. W2 · Logical indexing with [ ]
am == 0 compares every element with 0 and returns a **logical** vector of TRUE and FALSE
mpg[...]: keeps **only** the mpg values at the positions where the condition is TRUE
mean(): averages those values, so the line gives the **average** mpg of **automatic** cars
attach(mtcars) is what lets mpg and am be used by name
Key results: logical · only · automatic · average

11. W2 · A user-defined function: to12()
to12(): converts an hour on the **24-hour** clock, given as text, into a 12-hour label
as.integer(): turns the text "22" into the **number** 22 so it can be compared with 12 and reduced
paste(): **joins** its arguments with a space, so to12("22") returns "10 PM"
Flaw: hour > 12 sends 12 to the else branch, so noon is returned as "12 AM"
Key results: 24-hour · number · joins · 10 PM

12. W2 · Why as.integer() is needed
hour: "13" is a **character** string, and arithmetic is not defined for text
So hour - 12 stops with an **error** (non-numeric argument to binary operator), even though the comparison as.integer(hour) > 12 works
to12() first **converts** the input with as.integer(), so hour - 12 is computed on the number 13
Key results: character · error · as.integer

13. W2 · str_detect() with fixed()
str_detect(): returns **TRUE** or FALSE for each string, depending on whether it **contains** the pattern
fixed(): makes the pattern a **literal** string, matched character by character, not a regular expression
So hasMultipleLoc("Seoul, Suwon") is TRUE and hasMultipleLoc("Seoul") is FALSE
Key results: TRUE · contains · literal

14. W2 · sapply()
sapply(): calls to12() on **each element** of data$openingHour and simplifies the results into a **vector**
It is needed because to12() uses **if**, which takes a single TRUE or FALSE, so calling to12() on the whole column at once fails
Key results: each element · vector · if

15. W2 · ifelse()
ifelse(): checks a **condition** for **each element** and returns the second argument where it is TRUE and the third where it is FALSE
Here the TRUE or FALSE from hasMultipleLoc() becomes "Yes" or "No", one per café, and is stored in the new column hasMultipleLoc
Key results: condition · each element · Yes

16. W2 · read_excel(), as.data.frame() and colnames()
read_excel(): from the readxl package, reads the **Excel** file Café.xlsx into a **tibble**
as.data.frame(): converts the tibble into an ordinary base R data frame
colnames(data) = c(...): **renames** the four columns to type, location, openingHour and closingHour
Key results: Excel · tibble · rename

17. W5 · lm() and abline() with a model
lm(): fits a **linear model** by OLS, perform on eye, and returns a model object that holds the coefficients
abline(): given that object, draws the **fitted regression line** (its intercept and slope) on the scatterplot, in blue
The model is **stored** in eye_regr so that later calls such as summary() and confint() can use it
Key results: linear model · regression line · stores

18. W5 · summary() of an lm model
summary(): for an lm object reports the **coefficient table**, with Estimate, Std. Error, t value and p-value for each term
It also gives the residual standard error, the **R-squared** and adjusted R-squared, and the overall **F-statistic** with its p-value
The $r.squared part extracts only the R-squared from that summary
Key results: coefficient table · t value · R-squared · F-statistic

19. W5 · confint() and its level argument
confint(): returns a **confidence interval** for each **coefficient** of the model, one row per term with a lower and an upper bound
By **default** the level is 95% (columns 2.5 % and 97.5 %)
level: 0.99 gives 99% intervals (columns 0.5 % and 99.5 %), which are **wider**
Key results: confidence interval · coefficient · 95% · wider

20. W5 · cor() and round()
cor(): computes the Pearson **correlation** between perform and skill, and ^2 squares it
In a simple regression the squared correlation equals the **R-squared**, which is what the script checks
round(): rounds the difference to 16 decimal places so that tiny **floating**-point differences between the two calculations are not mistaken for a real gap
Key results: correlation · R-squared · floating

21. W5 · predict() with interval = "confidence"
predict(): plugs the **new data**, a data frame with the same variable names as the model, into the fitted equation and returns the **predicted** values (about 0.3475 and 0.3441)
interval: "confidence" adds the **lower and upper** bounds, so the result is a matrix with the columns fit, lwr and upr
level: 0.95 sets the confidence level of those bounds
Key results: predicted · new data · lwr · 0.3475

22. W5 · seq(), polygon() and lines()
seq(): creates 10,000 **evenly spaced** x-values from min(eye) - 3 to max(eye) + 3
polygon(): joins the upper bounds (reversed) and the lower bounds into one closed shape and **fills** it in transparent grey, the **shaded** confidence band
lines(): draws the upper and lower bounds as green **dashed** lines
Key results: evenly spaced · fills · dashed

23. W5 · set.seed() and rnorm()
set.seed(): fixes the starting point of the random number generator, so the same random numbers, and the same results, are **reproducible** on every run
rnorm(): draws sampleSize values from a **normal** distribution, here with **mean** 10 and **standard deviation** 2 for fertilizer
The second rnorm(sampleSize, 0, 1) adds random errors with mean 0 and standard deviation 1
Key results: reproducible · normal · mean 10 · standard deviation

24. W5 · The for loop and the coefficient matrix
for: **repeats** the body 10,000 times, once per rowIdx, each time drawing a new sample and fitting lm()
summary(...)$coefficients: is the coefficient matrix, and row 2 is fertilizer
coef[2,1]: the slope **estimate**, coef[2,3] its **t value** and coef[2,4] its **p-value**, each stored at position rowIdx of its vector
Key results: repeats · estimate · t value · p-value

25. W5 · ifelse(), table() and barplot()
pValue <= alpha gives TRUE where the p-value is at most 0.05
ifelse(): turns TRUE and FALSE into the **labels** "Reject" and "Fail to Reject"
table(): **counts** how many of the 10,000 samples fall under each label
barplot(): draws one **bar** per label with its count
Because the true slope is 0.2, not 0, the share of Reject estimates the test's **power**
Key results: labels · counts · bar · power

26. W5 · Filling a results data frame
data.frame(): with index = 1:sampleCnt creates a **data frame** with 20 rows
results[rowIdx, "slope"] = ... writes into one **cell**, chosen by row number and column name, and creates the column the first time it is used
interval["fertilizer", "2.5 %"] picks the **lower** bound of the fertilizer interval from the confint() matrix, and "97.5 %" the upper bound
&& is TRUE only if **both** conditions hold, so inside is TRUE when the true slope lies within the interval
Key results: data frame · cell · lower · both

27. W5 · ggplot() layers
ggplot(): sets the data (results) and the mapping of slope to the x-axis and index to the y-axis
geom_point(): draws each estimated slope as a **point**, one row per sample
geom_errorbarh(): draws a horizontal **error bar** from lower_bound to upper_bound, the confidence interval
geom_vline(): adds a green **vertical line** at the true slope 2, so the intervals that miss it are easy to see
Key results: point · error bar · vertical line

28. W6 · as.factor() inside lm()
as.factor(): turns session into a **factor**, a categorical variable, instead of a number
lm(): then creates a **dummy** variable for every category except one, the **base** (reference) category, which is absorbed in the intercept
Each dummy coefficient is the difference from the base category, holding x constant
Key results: factor · dummy · base

29. W6 · The : and * operators in a formula
x1:x2 adds only the **interaction** term, the product of x1 and x2
x1*x2: is shorthand for x1 + x2 + x1:x2, so it adds **both** the **main effects** and the interaction
x1 + x2: has the main effects only
Key results: interaction · main effects · both

30. W6 · linearHypothesis()
linearHypothesis(): from the car package, tests linear **restrictions** on the coefficients of a fitted model
Here it tests "x1 = 0" and "x2 = 0" **jointly**, not one at a time
For an lm model it reports an **F-test**, with the residual sums of squares of the restricted and unrestricted models, F and its p-value
Key results: restrictions · jointly · F-test

31. W6 · anova() with two nested models
anova(): given two **nested** models, **compares** them with an **F-test**
It uses the residual sum of squares (**RSS**) of each model and the number of extra parameters, and reports F and its p-value
Here the unrestricted model interacts every term with female, so the test is the **Chow** test of whether one regression fits both groups
Key results: compares · F-test · RSS · Chow